Software Requirements Specification Template

Software Engineering

The following annotated template shall be used to complete the Software Requirements Specification (SRS) assignment.

Template Usage:
Text contained within angle brackets (‘<’, ‘>’) shall be replaced by your project-specific information and/or details. For example, <Project Name> will be replaced with either ‘Smart Home’ or ‘Sensor Network’.

Italicized text is included to briefly annotate the purpose of each section within this template. This text should not appear in the final version of your submitted SRS.

This cover page is not a part of the final template and should be removed before your SRS is submitted.

Movie Ticketing System

Software Requirements Specification

Version1.2

10/22/26

Group 4
Jhef Grover Quiambao
Michael Farella
Dillon Holden
Zhoubin Nazari Boukani

Prepared for
CS 250- Introduction to Software Systems
Instructor: Gus Hanna, Ph.D.
Fall 2023

Revision History

Date	Description	Author	Comments
10/08/26	Version 1.1	Michael Farella	Second Revision
10/08/26	Version 1.1	Jhef Grover Quiambao	Second Revision
10/08/26	Version 1.1	Dillon Holden	Second Revision
10/08/26	Version 1.1	Zhoubin Nazari Boukani	Second Revision
10/22/26	Version 1.2	Michael Farella, Jhef Grover Quiambao, Dillon Holden, Zhoubin Nazari Boukani	Second Revision

Document Approval

The following Software Requirements Specification has been accepted and approved by the following:

Signature	Printed Name	Title	Date
	Michael Farella, Jhef Grover Quiambao, Dillon Holden, Zhoubin Nazari Boukani	Software Eng.	
	Dr. Gus Hanna	Instructor, CS 250	

Table of Contents

Revision History ii

Document Approval ii

1. Introduction 1

1.1 Purpose 1
1.2 Scope 1
1.3 Definitions, Acronyms, and Abbreviations 2
1.4 References 2
1.5 Overview 2

2. General Description 3

2.1 Product Perspective 3
2.2 Product Functions 3
2.3 User Characteristics 3
2.4 General Constraints 4
2.5 Assumptions and Dependencies 4

3. Specific Requirements 4

3.1 External Interface Requirements 5
3.1.1 User Interfaces 5
3.1.2 Hardware Interfaces 5
3.1.3 Software Interfaces 5
3.1.4 Communications Interfaces 5
3.2 Functional Requirements 6
3.2.1 Allow Ticket Selection and Purchase 6
3.2.2 Display Films and Reviews 8
3.2.3 Modify Showtimes, Seats, and Reviews 9
3.2.4Collect Customer Feedback 12
3.3 Use Cases 12
3.3.1 Purchase Movie Tickets 12
3.3.2 Modifying The Availability of a Seat 13
3.3.3 Creating a Feedback Ticket 14
3.4 Classes / Objects 16
3.4.1 Customer 16
3.4.2 Administrator 16
3.4.3 Movie 16
3.4.4 Theater 16
3.4.5 Auditorium 16
3.4.6 Showtime 17
3.4.7 Seat 17
3.4.8 Ticket / Booking 17
3.4.9 PromoCode/ Discount 17
3.4.10 Payment 17
3.4.11 Review 17
3.4.12Feedback 17
3.4.13 AuditLog 18
3.5 Non-Functional Requirements 18
3.5.1 Performance 18
3.5.2 Reliability 18
3.5.3 Availability 18
3.5.4 Security 19
3.5.5 Maintainability 19
3.5.6 Portability 19
3.6 Inverse Requirements 19
3.7 Design Constraints 19
3.8 Logical Database Requirements 19

3.9 Other Requirements 19

4. Analysis Models 20

4.1 Sequence Diagrams 20
4.3 Data Flow Diagrams (DFD) 20
4.2 State-Transition Diagrams (STD) 20

5. Change Management Process 20

6. Software Design Specification 20

6.1 System Description 21
6.2 Software Architecture Overview 21
6.3 UML Class Design 23
6.4 Development Plan and Timeline 27

7. Test Plan 30

7.1 Overview 30
7.2 Scope and Objectives 31
7.3Test Strategy 33
7.4Test Cases and Scenarios 33
7.4.1 Test Case Summary 33
7.4.2 Test Case Detail 34
7.5 Resources 35
7.6 Risk Assessment and Mitigation 35
7.7 Success Criteria 36
7.8 Deliverables and Sign-Off 36
7.9 Github Repository 37

8. Data Management Strategy 30

8.1 Overview and Goals 37
8.2 Data Stores and Number of Databases 38
8.3 Logical Organization of the Data 38
8.3.1 Logical Data Model 39
8.4 Partitioning by Theater 41
8.5 Consistency, Seat Holds, and Booking Transactions 41
8.6 Protecting Sensitive Data 42
8.7 Retention, Backup, and Recovery 42
8.8 Deliverables and Sign-OffDesign Decisions 43
8.9 Alternatives and Tradeoffs 44

A. Appendices 46

A.1 Appendix 1 46
A.2 Appendix 2 37

1. Introduction

The purpose of this Software Requirement Specification is to provide an overview of the requirements for the Movie Ticketing System. The Movie ticketing system is browser based and allows customers to view movies with their showtimes and are able to purchase tickets online or in person by using a self service terminal. Both forms of access will use the same information and update ticket availability.The detailed requirements of the Movie Ticketing System are provided in this document.

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to describe the requirements for the Movie Ticketing System. This document explains what the system should do, who will use it, and the conditions under which it will operate.

The intended audience includes the software development team, theater administrators, and the client. The document will be used as a guide when designing and developing the system.

The main purpose of the software is to provide customers with a way to purchase movie tickets while also giving theater administrators tools to manage movies, showtimes, and ticket information.

1.2 Scope

The software product being developed is the Movie Ticketing System. It will be a browser-based system that allows customers to view movies, showtimes, and ticket availability and purchase tickets online or through a self-service terminal at the theater. Both forms of access will use the same information and update ticket availability in real time.

The system will allow customers to purchase tickets, use available discounts, and select seats when assigned seating is available. Customers will be limited to a maximum of 20 tickets in a single transaction.

The system will also provide an administrator mode that allows theater employees to manage showtimes, theaters, and customer purchasing issues.

The main goal of the system is to provide customers with a faster and more reliable way to purchase movie tickets while keeping ticket information updated between the website and theater terminals. The system is intended to improve the current ticketing system, which the client described as slow and has some bugs.

The current version will focus on theater locations in the San Diego area. A feature for showing nearby theaters may be added in a later version and is not included in the current scope.

1.3 Definitions, Acronyms, and Abbreviations

SRS	*Software Requirements Specification.*
Customer	*A person who uses the system to view movies, showtimes, and purchase tickets.*
Administrator	*A theater employee who can manage showtimes, theater information, and customer purchasing issues.*
Self Service Terminal	*A terminal at the theater that allows customers to purchase tickets in person.*
Showtime	*The scheduled time when a movie begins.*
Auditorium	*An individual theater room.*
Deluxe Theater	*A theater with assigned seating.*
Regular Theater	*A theater without assigned seating.*

1.4 References

Theater Ticketing System Reqs, project requirements document.

Theater Ticketing System Qs, client interview and project questions.

Theater Ticketing Requirements, functional and non-functional requirements document.

Theater Ticketing Questions — Additional client questions and requirements.

1.5 Overview

The remainder of this Software Requirements Specification is organized into several sections. Section 2 provides a general description of the Movie Ticketing System, including the product perspective, product functions, user characteristics, general constraints, and assumptions and dependencies. Section 3 describes the specific requirements of the system, including interfaces, functional requirements, use cases, classes and objects, and non-functional requirements. Section 4 contains the analysis models used for the system, and Section 5 explains the process for managing changes to the requirements.

2. General Description

The Movie Ticketing System will provide customers and theater administrators with a browser based system for viewing movies, checking showtimes and ticket availability as well as purchasing tickets, and managing theater information. This section describes the overall characteristics, functions, users, constraints, assumptions, and dependencies of the system.

2.1 Product Perspective

The Movie Ticketing System is intended to improve the theater's current ticketing system. The new system will be browser based and will allow customers to purchase tickets online or through a self service terminal at the theater. Both methods of access will use the same information and update ticket availability. All theater locations will use one central database that contains information about each different theater including showtimes and available tickets.

2.2 Product Functions

Movie Ticketing System will provide customers and administrators with several functions. Customers will be able to view movies with their showtimes, check ticket availability, and purchase tickets online or through a self service terminal. Customers will be limited to a maximum of 20 tickets in a single transaction.

The system will support different discounts for tickets such as student, military, and senior customers. Customers will also be able to select assigned seats in deluxe theaters while regular theaters will use open seating.

The system will support customer accounts, payment processing, customer feedback, and multiple languages. Customers can create an optional loyalty account that stores information such as payment history, payment information, and loyalty points.

Administrators will be allowed access to an administrator mode that allows them to manage showtimes, change theater information, and assist customers when purchasing problems occur.

2.3 User Characteristics

Movie Ticketing System will have two types of users, customers and administrators.

Customers may access the system online through a web browser or in person through a self service terminal at the theater. Customers will use the system to view movies and showtimes, purchase tickets, and receive digital or physical tickets. Customer accounts will be optional.

Administrators will be theater employees who are given additional access to the system. They are allowed to manage showtimes, theater information, and correct customer purchasing problems when needed.

The user interface should be ergonomic and easy to use so that customers can complete ticket purchases without unneeded difficulty.

2.4 General Constraints

The System must run through a web browser and not as a mobile application.

The system will initially support 20 theater locations in the San Diego area and will operate using the Pacific time zone. Customers may purchase a maximum of 20 tickets in a single transaction. Tickets may be purchased up to two weeks prior to a showing and up to 10 minutes after a scheduled showtime begins.

Once a customer starts the purchasing process, the selected tickets will only be held for five minutes before they are released.

The client expects a prototype within four months and has given an initial budget of $500,000 for the project

2.5 Assumptions and Dependencies

The Movie Ticketing System depends on the central database to provide information about theaters, showtimes, and available tickets. Client stated that one database will be used for all theaters and partitioned to all the different theaters.

The system depends on payment services to process ticket purchases. Credit card, PayPal, and Bitcoin can be accepted as payment methods

The system may also depend on outside movie review services because the client wants the system to display ratings or critic quotes from sources such as Rotten Tomatoes or IMDb.

It is assumed that customers using the online system or theater terminals will have access to the network in order to communicate with the central ticketing database.
The system will depend on the central theater database for movie, showtime, and ticket information. Digital ticket delivery will depend on email when customers choose to receive their tickets electronically.

3. Specific Requirements

This will be the largest and most important section of the SRS. The customer requirements will be embodied within Section 2, but this section will give the D-requirements that are used to guide the project's software design, implementation, and testing.

Each requirement in this section should be:

- *Correct*
- *Traceable (both forward and backward to prior/future artifacts)*
- *Unambiguous*
- *Verifiable (i.e., testable)*
- *Prioritized (with respect to importance and/or stability)*
- *Complete*
- *Consistent*
- *Uniquely identifiable (usually via numbering like 3.4.5.6)*

Attention should be paid to the carefuly organize the requirements presented in this section so that they may easily accessed and understood. Furthermore, this SRS is not the software design document, therefore one should avoid the tendency to over-constrain (and therefore design) the software project within this SRS.

3.1 External Interface Requirements

This section describes how the Movie Ticketing System will interact with users, hardware, other software, and communication services.

3.1.1 User Interfaces

The Movie Ticketing System will provide a browser-based user interface that can be accessed online or through a self-service terminal at the theater. Both interfaces will use the same information and update ticket availability.

Customers will be able to use the interface to view movies and showtimes, check ticket availability, purchase tickets, and select seats when assigned seating is available.

The system will also provide an administrator interface that allows authorized theater employees to manage showtimes, change theater information, and assist customers with purchasing problems.

The customer interface will support English, Spanish, and Swedish.

3.1.2 Hardware Interfaces

The Movie Ticketing System will be accessible from devices capable of running a supported web browser. Customers may also use self-service terminals located at theater locations to purchase tickets in person.

The system must support physical printable tickets as well as digital tickets.

The project requirements do not currently specify exact hardware models, printer types, scanners, or other physical device specifications.

3.1.3 Software Interfaces

The Movie Ticketing System will interact with the central theater database to access information about theaters, movies, showtimes, and ticket availability. The client stated that one database will be used for all theaters and partitioned among the different theater locations.

The system will also interact with payment services to process ticket purchases. Supported payment methods include credit card, PayPal, and Bitcoin.

The system may also interact with external movie review services to display movie scores and critic quotes from sources such as Rotten Tomatoes or IMDb.

Email services will be used when customers choose to receive digital tickets through email.

3.1.4 Communications Interfaces

The Movie Ticketing System will require communication between the online website, theater self-service terminals, and the central ticketing database so that movie, showtime, and ticket availability information remains updated across the system. The client requires the online and in-person systems to show the same data and update it live.

The system will also communicate with external payment services and email services when processing purchases and delivering digital tickets.

3.2 Functional Requirements

This section describes specific features of the software project. If desired, some requirements may be specified in the use-case format and listed in the Use Cases Section.

3.2.1 Allow Ticket Selection and Purchase

3.2.1.1 Introduction

This feature enables users to select specific seats for a chosen movie showtime, apply promotions and discounts, and complete the purchase via an online payment gateway.

Primary Actor: Customer

Preconditions:

1. The selected showtime must have available seats. The seats shall only be available given the following criteria:
    a. The seats have not been sold or placed on hold.
    b. The seats are not made unavailable by the administrator.
    c. The showtime is within 2 weeks.
2. The system must have a connection to the payment gateway API.
3. The user must be on the seat selection page.

3.2.1.2 Inputs

Showtime ID: Unique identifier for the selected screening

Selected Seats: Array of seat identifiers (eg. [“C12”, “D8”]), requires (Min:1 , Max: 20)

Ticket Types: Objects to specify ticket categories in order to apply discounts (e.g, Adult, Senior, Military)

Promo Code: Optional alphanumeric string for applying discounts

Payment Details: Handles users credit card/ payment information to pass to the payment API

Contact Details: Input for email and phone number to allow guest checkout

User Age: An integer to collect user age to verify users are old enough to purchase age restricted tickets (eg. Age >= 17 : R films, Age >= 18 : NC-17 films)

Trigger: A “Complete Purchase” button shall begin the processing.

3.2.1.3 Processing

1. Seat Reservation Holding: Seat selection shall place a temporary five-minute hold on the selected seat IDs to prevent double booking.
2. Price Calculation: The system shall calculate the total order price and shall calculate the subtotal as sum of the product of the ticket quantity and ticket category price. Valid

promo codes shall apply a discount to the subtotal. Local sales tax should be applied to the potentially discounted subtotal.

3. Payment Authorization: The system shall transmit the encrypted payment token and calculated grand total to the third-party payment gateway API.
4. Order Finalization: A successful authorization from the payment gateway shall do the following:
    a. Update the seat holding to a sold ticket.
    b. Generate a booking reference code.
    c. Generate a QR code for encoding the booking reference code and ticket details.
    d. The system shall write the transaction details to the systems database.

3.2.1.4 Outputs

UI Response: The system shall redirect the user to an order confirmation page displaying:

a. A booking reference code
b. Showtime details including theater location, auditorium, date, showtime, assigned seat number, and scannable QR for entry.
c. An itemized price breakdown

Data State: The system shall update the seat records and store them to the showtime and ticket database.

External Notification: The system shall send a transaction confirmation email containing the QR code and ticket details to email specified by the contact details or email address associated with the user's account.

3.2.1.5 Error Handling

Scenario 1: Seat Hold Expiration

Condition: The five-minute timer expires before payment is complete.

System Action: The system shall release the seat hold, clear the payment form and flag the transaction as expired.

User Feedback: The system shall display a notification to the user that the session has timed out, and to reselect their seats.

Scenario 2: Concurrent Seat Selection

Condition: Another user completes checkout for one or more selected seats while the current user is on the available seat map.

System Action: The system shall end the current transaction hold.

User Feedback: The system shall display a transaction error message stating which selected seats are no longer available, and to select alternative seats instead.

Scenario 3: Payment Decline

Condition: The payment gateway returns a refusal code.

System Action: The system shall maintain the hold timer and log the failed transaction attempt.

User Feedback: The system shall display a message indicating that the payment processing failed and to verify payment details or provide another payment option.

Scenario 4: Invalid/Expired Promo Code

Condition: The entered promo code is not recognized or is recognized as invalid.

System Action: The system shall recalculate order subtotal without discount.

User Feedback: The system shall display an error message informing the user the promo code is invalid.

Scenario 5: Underage User

Condition: The user is not old enough to purchase tickets for an age restricted screening.

System Action: The system shall store an age verification failure.

User Feedback: The system shall notify the user they do not fulfill the age requirement for the film.

3.2.2 Display Films and Reviews

3.2.2.1 Introduction

This feature enables users to view, search, and filter for showtimes with different experience formats (deluxe theater options e.g(2D, 3D, IMAX)) and simultaneously displays online reviews from external film review sources.

Primary Actor: Guest User, Registered Guest

Preconditions:

1. The system shall have an active movie schedule as permitted by the content management system.
2. The system must periodically ingest, parse, and store external movie reviews into a scraped review database.

3.2.2.2 Inputs

Selected Date: The system shall accept a date value to display films and showtimes associated with that date.

Movie Filter: An Array of Movie ID's for filtering searches

Experience Format Filter: An enum for (2D, 3D, IMAX, etc)

Trigger: The user navigates to showtimes.

3.2.2.3 Processing

1. The system shall set default parameters configurable by the admin.
2. The system shall query for active screens matching user set filter criteria.
3. The system shall display the results of the query based on the active filter(s).
4. The system shall display with each film an aggregated score (eg, 0-5 stars, 1-10 rating), review count, and top/ most recent review snippet.
5. The system shall calculate seat availability from the database and display screenings as sold out or nearly sold out.
6. The system shall sort showtimes chronologically from earliest to latest start time within each format group.

3.2.2.4 Outputs

UI Response: The system shall render an interactive schedule based on the showtimes for the next two weeks. It shall display:

a. Movie cards featuring:
    a. Poster Art, Title, MPAA rating, Genre tags, and runtime
    b. Online review badge: Displays aggregated rating and review count
    c. Review Snippet Drawer: an expandable section allowing users to read top/ recent reviews

b. Format badges: (eg.IMAX, 3D)

c. Interactive buttons displaying start time and seat availability state.

Data State: None, the output is read only.

3.2.2.5 Error Handling

Scenario 1: No Showtimes Found

Condition: The query returns 0 records for the selected combination of filters.

System Action: The system shall maintains the filter UI controls

User feedback: The system shall display an empty state message, informing the user to adjust their filters.

Scenario 2: Missing or Outdated Scraped Review Data

Condition: There is no scraped review record for a film or reviews available within the configured threshold.

System Action: The system shall fallback and render the movie card normally without interrupting page execution.

User Feedback: The system shall display a status placeholder instead of the review such as "Review Pending".

3.2.3 Modify Showtimes, Seats, and Reviews

3.2.3.1 Introduction

The feature enables authorized administrative personnel to manage schedule listings, modify theater seating availability/layout and override and moderate online review data.

Primary Actor: Administrative User

Preconditions:

1. Admin users must be authenticated with an active session with valid administrative privileges.

3.2.3.2 Inputs

Action Category: Specification of what to change (Movie Management, Showtime Management, Auditorium Management, Seating Management, Review Management)

Action Type: The system provides the kind of action. (Create, Update, Delete/Archive)

Movie Parameters: (e.g Movie Title, Description, Runtime MPAA)

Showtime Parameters: (Showtime ID, Movie ID, Auditorium ID, Start Time Stamp, Base Pricing)

Seating Parameters: (Seat Identifiers, Seat Status Flag (Available, Maintenance/Blocked, VIP)

Review Moderation Parameters (Review ID, Review Text Snippet, Visibility Status (eg. Published, Hidden, Flagged)

Trigger: The system shall allow the admin to click to "Save Film", "Schedule Showtime" or "Publish Overrides" on an admin portal.

3.2.3.3 Processing

1. Access Verification: The system shall verify that the requesting user possesses admin permissions before any creation of modification.
2. Movie Catalog Processing:
    a. The system shall check for the database to ensure that newly added films are not within the database.

   b. The system shall create a unique Movie ID and pending scrape status upon creation of a new film within the database.
3. Showtime Modification and Validation:
   a. The system shall confirm that the assigned Movie ID exists and has an active status within the catalog.
   b. The system shall calculate total screening duration by adding a customizable buffer.
   c. The system shall execute an overlap check against the existing schedule for the same Auditorium ID.
4. Seating Layout and Status Processing:
   a. The system shall verify the current booking state of the target seat.
   b. System shall block status changes (eg, setting maintenance/blocked for seats flagged as sold) unless an admin override flag is provided.
5. Review Data Moderation Processing:
   a. The system shall allow the admin to modify or hide a scraped review and marks it as overridden.
   b. The system shall recalculate a film's aggregate rating using updated review values as necessary.
6. Persistence and Cache Invalidation:
   a. The system shall commit changes to the showtimes, seats, and reviews tables within the database.
   b. The system shall invalidate the application layer cache for public showtimes to reflect the changes live.
7. Audit Logging:
   a. The system shall record admin audit logs including their admin id, changes , timestamp, and reason for audit.

3.2.3.4 Outputs

UI Response:

a. The system shall display a confirmation message of the changes made.
b. The system shall refresh the admin catalog tables and calendar.

Data State: The system shall commit new records; the database tables updated; and the audit log entries committed.

3.2.3.5 Error Handling

Scenario 1: Duplicate Movie Entry

Condition: Admin attempts to add a film with a title and year already present in the movies table.

System Action: The system shall end the movie record creation

User Feedback: The system shall display an error message informing the user of the duplicate entry.

Scenario 2: Modification of Occupied Seats

Condition: An admin attempts to set seat status to maintenance/blocked of a seat that has already been sold.

System Action: The system pauses the transaction and requires a secondary confirmation to override. The system also triggers a process to refund the customer.

User Feedback: The system prompts the administrator to continue with the action/

Scenario 3: Unauthorized Access

Condition: Admin session expires or user token lacks sufficient scope

System Action: Block the transaction attempt and invalidate the session token

User Feedback: The system shall redirect the user to the admin login page with a message stating the session either expired or was unauthorized.

3.2.4 Collect Customer Feedback

3.2.4.1 Introduction

This feature enables the user to submit feedback, star ratings, and optional comments regarding their experience and platform usability.

Primary Actor: Customer

Preconditions:

1. The user is accessing the feedback portal.
2. The system shall be configured to store customer feedback.

3.2.4.2 Inputs

Customer ID/ Email: An optional input for guests

Booking Reference Code: String to identify purchase, optional

Feedback Category: A value to categorize the kind of feedback (eg. Service/Theater feedback, website usability, other)

Feedback Rating: A 1 to 5 integer to describe feedback

Media Attachment: An image file upload, optional

Trigger: Customer clicks submit feedback

3.2.4.3 Processing

Input Validation and Sanitization: The system shall validate a category and numeric rating. The system shall sanitize the comment to prevent code injection.

Booking Verification: The system shall compare Booking Reference Code to the database and mark feedback as verified.

Spam and Rate Limiting: The system shall restrict feedback to 3 entries per IP address within a 24 hour window.

Escalation Routing: The system shall escalate verified reviews under 2 stars and flag the review as high priority. The system shall also generate an alert notification.

3.2.4.4 Outputs

UI Response: The system shall display a confirmation message.

Data State: The system shall insert a new record of customer feedback into the database with a timestamp status, and priority level.

Notification: The system shall dispatch notifications to administrators for low-ratings and flagged feedback entry.

3.2.4.5 Error Handling

Scenario 1: Missing Required Fields

Condition: User attempts submission without selecting a rating or feedback category

System Action: Abort the submission attempt and highlight the missing form field

User Feedback: The system shall display an error message to the user.

Scenario 2: Invalid Booking Reference

Condition: The provided booking reference does not match a value in the system

System Action: Prompt the user to proceed as general feedback

User Feedback: The system shall alert the user the Booking code is not recognized and prompts the user to either reenter the booking code or submit as general feedback.

3.3 Use Cases

3.3.1 Use Case #1 Purchase Movie Tickets

Actor: Guest

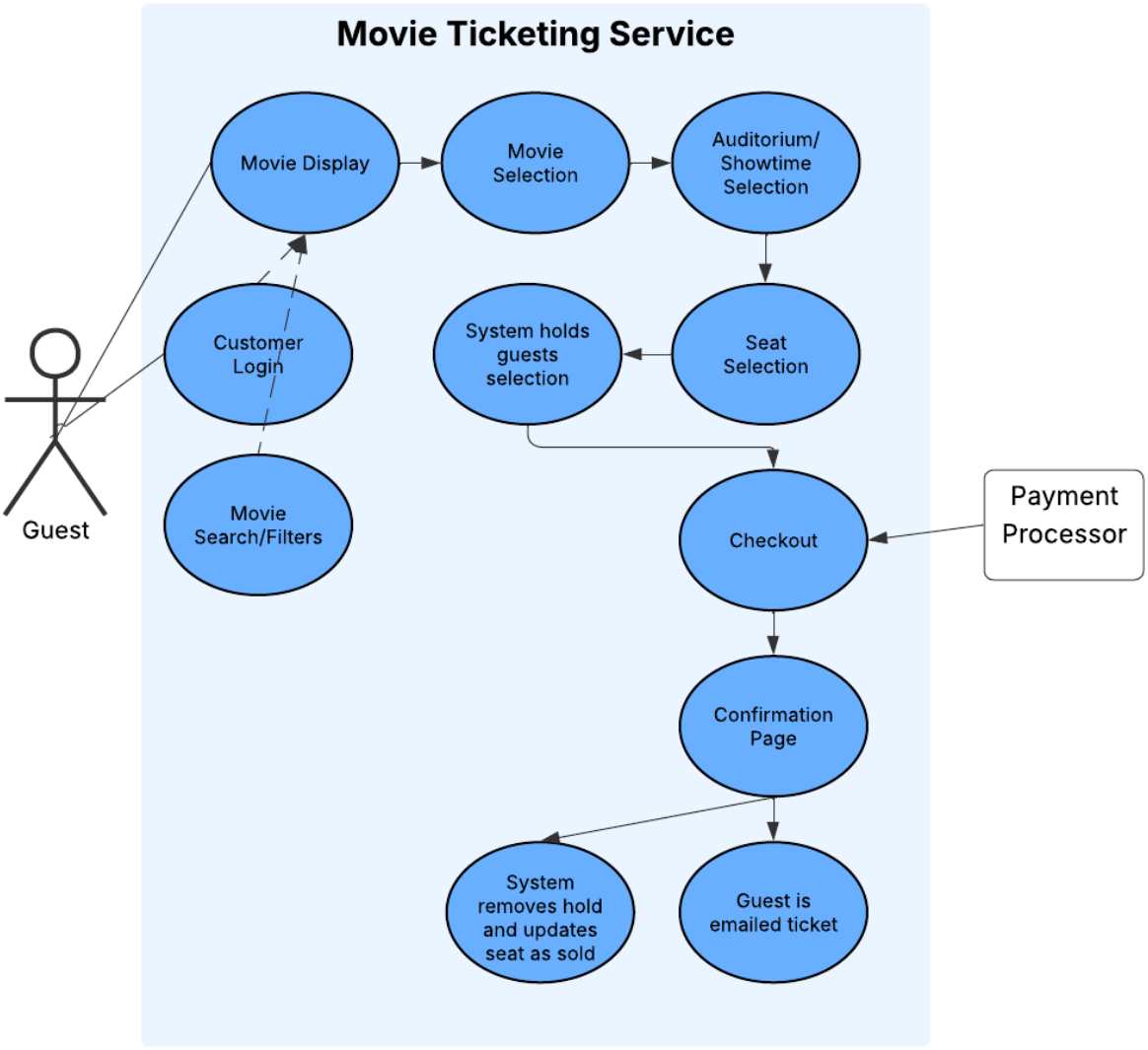
Flow of events:

1. The user connects to the movie ticketing site.
2. The user chooses to login or continue with guest checkout
3. The user accesses available films
4. The user chooses to utilize the search function to filter for a specific movie or browses the default option.
5. The system provides available movies, showtimes, and seats.
6. The user selects a film.
7. The user selects a showtime.
8. The user selects a seat.
9. The system creates a hold for the user's purchase.
10. The user provides payment information.
11. A payment processor handles the transaction.
12. The user is provided with a confirmation page and email.

Entry condition:

1. The user must be identified as a real user and not a bot.
2. The user must be using a supported browser.

Use Case Diagram:



3.3.2 Use Case #2 Modifying The Availability of a Seat

Actor: Admin

Flow of events:

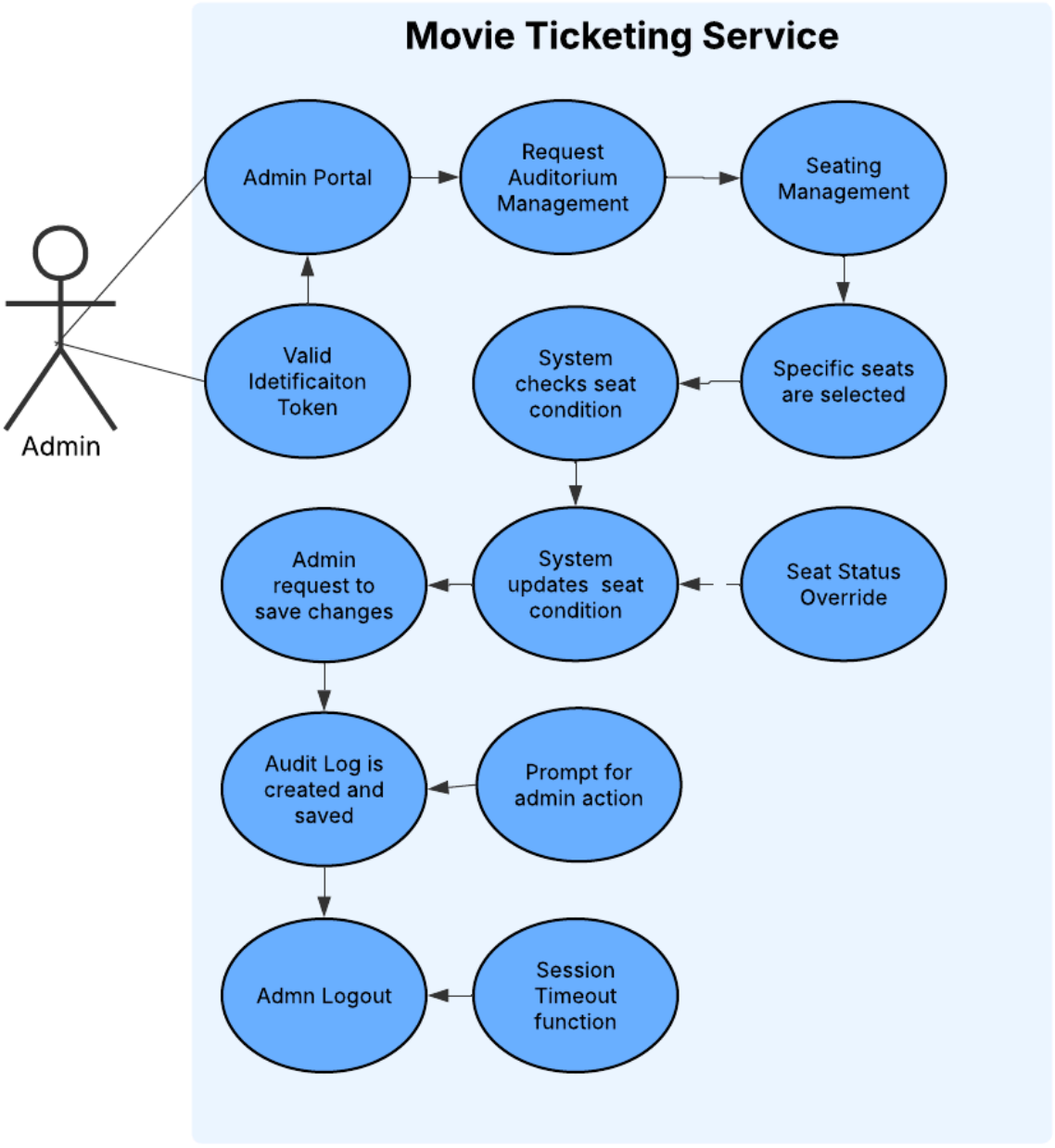
1. The admin connects to an admin login portal.
2. The admin provides a valid login token.
3. The admin requests to manage an auditorium.
4. The admin requests to modify a seating layout.
5. The admins request to modify a specific array of seats.
6. The system checks whether or not the seats have been sold or are available.
7. If the seats are sold, the admin is prompted to override the seat status. If the admin overrides the seats a refund request process is initiated.
8. The admin saves changes.

9. The system creates an audit log and requests the reason for the administrative action.
10. The admin describes the administrative request.
11. The admin logs out of the portal.
12. The system terminates the session.

Entry Condition:

The administrator provides a valid token for accessing the portal.

Use Case Diagram



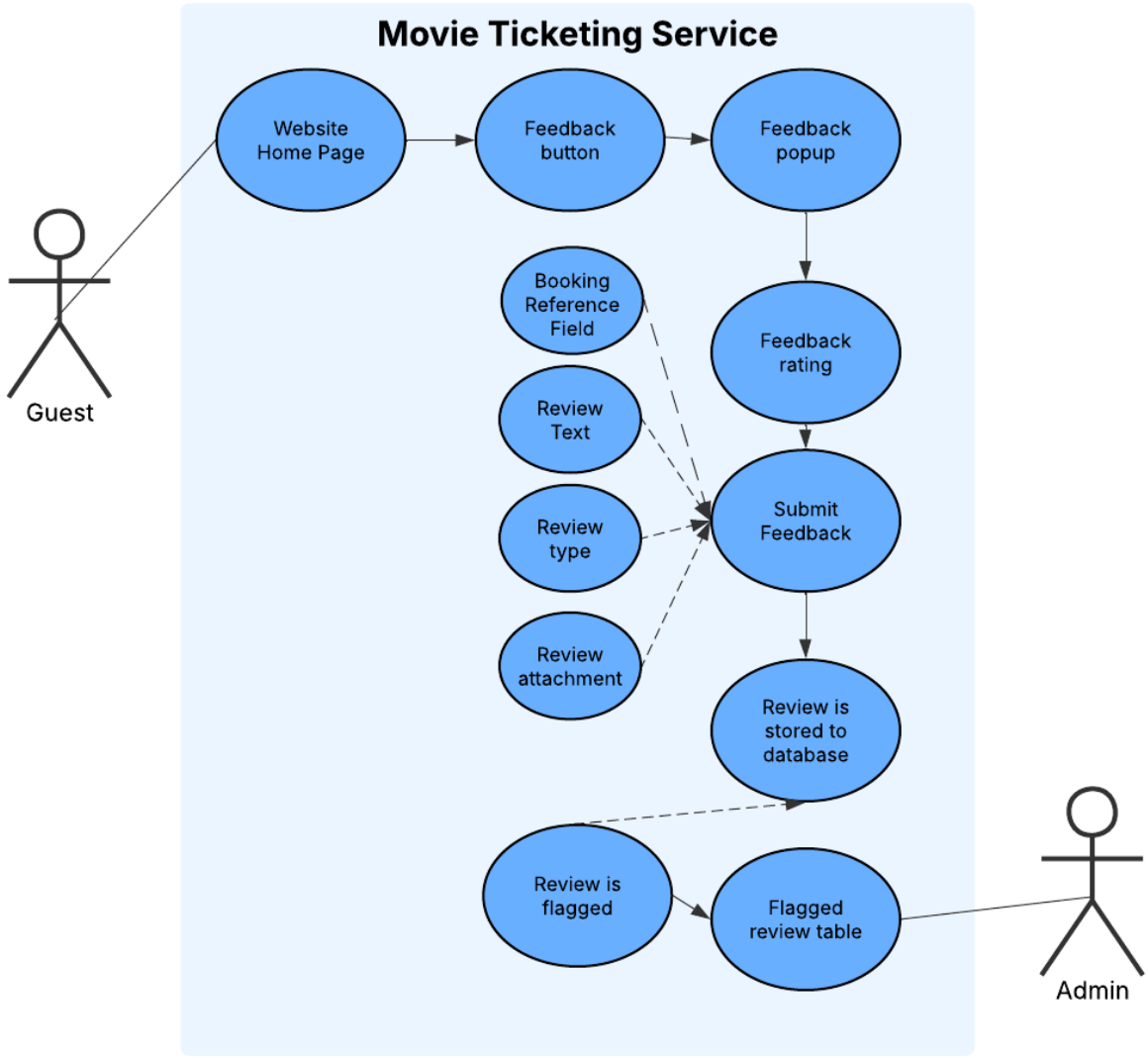
3.3.3 Use Case #3 Creating a Feedback Ticket

Actor: Guest

Flow of Events:

1. The user accesses the website
2. The user access the feedback page
3. The user provides a feedback rating
4. The user may provide a feedback category, file upload, and a booking reference code.
5. The user submits the feedback.
6. The user receives a confirmation of their feedback.
7. The system sanitizes the data imputed by the user.
8. The system evaluates the review and flags it as necessary.
9. The administrator is notified if a review is flagged by the system.

Use Case Diagram:



3.4 Classes / Objects

3.4.1 Customer

3.4.1.1 Attributes :customerID (optional, register accounts), name, email, phone number, password (hashed), preferredLanguage, accountType (Guest/Registered), paymentHistory, loyaltyPoints
3.4.1.2 Functions: login(), createAccount(), searchMovies(filters), selectSeats(), purchaseTickets(), submitFeedback(), viewOrderHistory()
Reference: 3.2.1 Allow Ticket Selection and Purchase; 3.2.4 Collect Customer Feedback; Use Case #1; Use Case #3

3.4.2 Administrator

3.4.2.1 Attributes: adminID, name, username, password (hashed), permissionLevel, sessionToken
3.4.1.2 Functions: login(), manageShowtimes(), manageAuditoriums(), manageSeats(), moderateReviews(), overrideSeatStatus(), viewAuditLog()
Reference: 3.2.3 Modify Showtimes, Seats, and Reviews; Use Case #2

3.4.3 Movie

3.4.3.1 Attributes: movieID, title, description, runtime, MPAA rating, genre tags, posterArt, releaseDate, scrapeStatus
3.4.3.2 Functions: addMovie(), updateMovie(), archiveMovie(), getAggregatedRating()
Reference: 3.2.2 Display Films and Reviews; 3.2.3.3 (Movie Catalog Processing)

3.4.4 Theater

3.4.4.1 Attributes: theaterID, name, address, timeZone, auditoriumList
3.4.4.2 Functions: getShowtimes(), getAuditoriums()
Reference: 2.1 Product Perspective; 2.4 General Constraints

3.4.5 Auditorium

3.4.5.1 Attributes: auditoriumID, theaterID, type (Deluxe/Regular), capacity, seatingLayout, supportedFormats (2D/3D/IMAX)
3.4.5.2 Functions: getSeatMap(), checkScheduleOverlap()
Reference: 1.3 Definitions (Deluxe/Regular Theater); 3.2.3.3 (Showtime Modification and Validation)

3.4.6 Showtime

3.4.6.1 Attributes: showtimeID, movieID, auditoriumID, startTimeStamp, endTimeStamp (start + buffer), basePricing, formatType, availableSeatCount
3.4.6.2 Functions: Reference: 3.2.2 Display Films and Reviews; 3.2.3 Modify Showtimes, Seats, and Reviews; Use Case #1

3.4.7 Seat

3.4.7.1 Attributes: seatID, auditoriumID, seatIdentifier (row/number), status (Available/Held/Sold/Maintenance/VIP), holdExpirationTimestamp
3.4.7.2 Functions: placeHold(), releaseHold(), markSold(), overrideStatus()
Reference: 3.2.1.3(1) Seat Reservation Holding; 3.2.3.3(4) Seating Layout and Status Processing; Use Case #2

3.4.8 Ticket / Booking

3.4.8.1 Attributes: bookingReferenceCode, customerID (optional), showtimeID, seatIDs[], ticketTypes[], subtotal, discountApplied, taxAmount, grandTotal, qrCode, transactionTimestamp, deliveryMethod
3.4.8.2 Functions: calculatePrice(), applyPromoCode(), generateBookingReference(), generateQRCode(), finalizeOrder()
Reference: 3.2.1 Allow Ticket Selection and Purchase (full); Use Case #1

3.4.9 PromoCode/ Discount

3.4.9.1 Attributes: code, discountType (Student/Military/Senior/Promotional), discountValue, expirationDate, validityStatus
3.4.9.2 Functions: validateCode(), applyDiscount()
Reference: 3.2.1.3(2) Price Calculation; 3.2.1.5 Scenario 4 (Invalid/Expired Promo Code)

3.4.10 Payment

3.4.10.1 Attributes: paymentID, paymentMethod (Credit Card/PayPal/Bitcoin), encryptedToken, authorizationStatus, amountCharged, timestamp
3.4.10.2 Functions: transmitToGateway(), receiveAuthorization(), logFailedAttempt()
Reference: 3.2.1.3(3) Payment Authorization; 3.5.4 Security

3.4.11 Review

3.4.11.1 Attributes: reviewID, movieID, source (Rotten Tomatoes/IMDb), reviewSnippet, rating, visibilityStatus (Published/Hidden/Flagged), scrapeTimestamp
3.4.11.2 Functions: ingestReview(), moderateReview(), recalculateAggregateRating()
Reference: 3.2.2 Display Films and Reviews; 3.2.3.3(5) Review Data Moderation Processing

3.4.12 Feedback

3.4.12.1 Attributes: feedbackID, customerID/email (optional), bookingReferenceCode (optional), category, rating (1–5), comment, mediaAttachment, priorityLevel, status, timestamp
3.4.12.2 Functions: submitFeedback(), validateInput(), sanitizeComment(), verifyBookingReference(), escalateIfLowRating()
Reference: 3.2.4 Collect Customer Feedback (full); Use Case #3

3.4.13 AuditLog

3.4.13.1 Attributes: logID, adminID, actionType, changeDescription, reason, timestamp

3.4.13.2
Functions : createLogEntry(), storeLogEntry(), accessLog()
Reference: 3.2.3.3(7) Audit Logging

…

3.5 Non-Functional Requirements

Non-functional requirements may exist for the following attributes. Often these requirements must be achieved at a system-wide level rather than at a unit level. State the requirements in the following sections in measurable terms (e.g., 95% of transactions shall be processed in less than a second, system downtime may not exceed 1 minute per day, > 30 day MTBF value, etc).

3.5.1 Performance

1. The system should load movie listings, showtimes, reviews, and seating charts within two seconds for at least 95% of customer requests.
2. The system should support at least 500 customers using the website at the same time without a major decrease in performance.
3. Seat availability should update within two seconds after a customer completes a ticket purchase.
4. After payment is approved, the customer should receive a purchase confirmation within five seconds.

3.5.2 Reliability

1. The system must prevent the same seat from being sold to more than one customer for the same movies and showtime.
2. All successfully completed purchases must be saved in the ticket database.
3. If two customers attempt to purchase the same seat, the seat will be given to the customer whose payment is approved first.
4. If a payment is declined or the seats become unavailable, the system must not complete the reservation or charge the customer.
5. The system should operate for at least 30 days between major system failures.

3.5.3 Availability

1. The system should be available at least 99% of each month, except during scheduled maintenance periods.
2. Scheduled maintenance should not last longer than two hours per month.
3. Customers should be notified at least 24 hours in advance before scheduled maintenance.
4. If the system unexpectedly stops working, ticket-purchasing services should be restored within 15 minutes.

3.5.4 Security

1. The system must encrypt personal and payment information while it is being transmitted.
2. Payments must be completed through a secure outside payment processor.

3. The system must not store customers' complete credit or debit card numbers.
4. Customers must enter a valid username and password before accessing their account information or purchase history.
5. The system should temporarily lock an account for 10 minutes after five unsuccessful login attempts.
6. Only authorized administrators may change movies, showtimes, discounts, ticket prices, and seat inventory.

3.5.5 Maintainability

1. Administrators should be able to update movies, showtimes, prices, discounts, and seat availability without changing the website's source code.
2. The system should keep a record of errors, including the date, time, and part of the system where each error occurred.
3. Ticket purchasing, seat selection, and payment functions must be tested before each software update is released.
4. System documentation must be updated whenever an existing feature is changed or a new feature is added.

3.5.6 Portability

1. The website should work on the two most recent versions of Chrome, Safari, Firefox, and Microsoft Edge.
2. Customers should be able to use the website on Windows, macOS, iOS, and Android devices.
3. The website should adjust to fit desktop, tablet, and smartphone screen sizes.
4. The website should support touchscreen, mouse, and keyboard controls.
5. Customers should not be required to install additional software or browser plug-ins to use the website.

3.6 Inverse Requirements

*State any *useful* inverse requirements.*

3.7 Design Constraints

Specify design constraints imposed by other standards, company policies, hardware limitations, etc. that will impact this software project.

3.8 Logical Database Requirements

Will a database be used? If so, what logical requirements exist for data formats, storage capabilities, data retention, data integrity, etc.

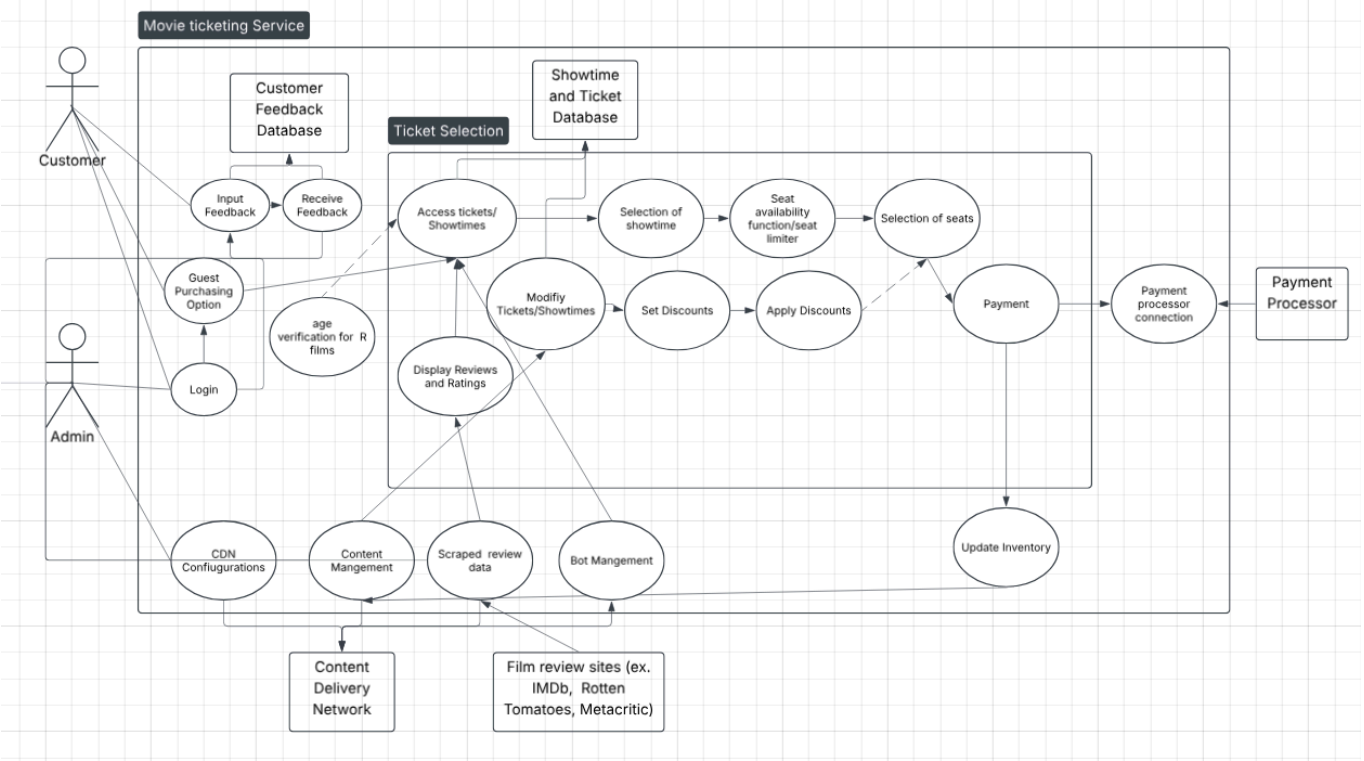
3.9 Other Requirements

Catchall section for any additional requirements.

4. Analysis Models

List all analysis models used in developing specific requirements previously given in this SRS. Each model should include an introduction and a narrative description. Furthermore, each model should be traceable to the SRS's requirements.

4.1 Sequence Diagrams



4.2 State-Transition Diagrams (STD)

4.3 Data Flow Diagrams (DFD)

5. Change Management Process

Identify and describe the process that will be used to update the SRS, as needed, when project scope or requirements change. Who can submit changes and by what means, and how will these changes be approved.

6. Software Design Specification

This section describes the planned software design for the Movie Ticket Star system. It provides an overview of the system, its software architecture, UML class design, major components, and development plan. The design is based on the functional and non-functional requirements documented in the previous sections of this SRS. Its purpose is to give the development team enough information to understand how the system's components will be organized and how they will interact during implementation.

6.1 System Description

Movie Ticket Star is a browser-based movie ticketing system that will support multiple theater locations in the San Diego area. Customers will be able to view movies and showtimes, check ticket availability, select seats when assigned seating is offered, and purchase up to 20 tickets in one transaction. Customers may access the system through the website or a self-service terminal at a theater. Both methods will use the same central database so that ticket and seat availability remain consistent.

The system will support guest checkout and optional customer accounts. Registered customers may view their purchase history and earn loyalty points. The system will also support eligible ticket discounts, multiple languages, digital or printed tickets, customer feedback, and payments through an outside payment processor.

Authorized theater administrators will use a separate administrator interface to manage movies, showtimes, auditorium information, seating availability, reviews, discounts, and customer purchasing problems. Changes made through the Administrator Portal will be stored in the Central Database and reflected across the Customer Website and Self-Service Terminals.

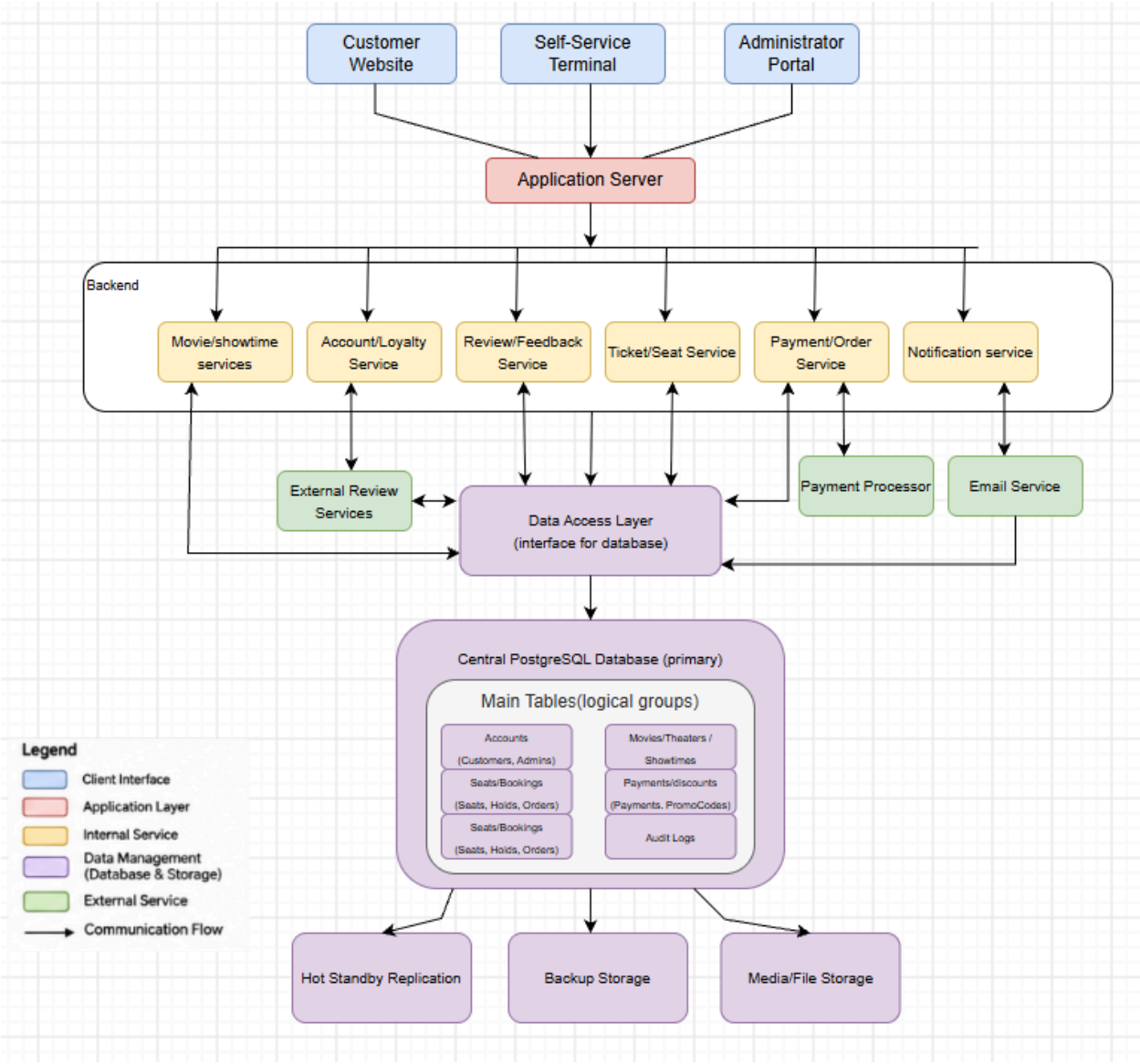
6.2 Software Architecture Overview

The Movie Ticket Star system will use a layered client-server architecture. This structure separates the user interfaces, application processing, database storage, and outside services. Separating the system into these areas will make it easier to maintain, test, and update individual parts of the system without requiring the entire application to be changed.

Customers and administrators will access the system through browser-based interfaces. Their requests will be sent to the application server, which will process each request through the appropriate system component. The application server will communicate with the central database and with outside services when payment processing, email delivery, or movie review information is required.

6.2.1 Software Architecture Diagram

The software architecture diagram below shows the major components of the Movie Ticket Star system and the connections between them.



6.2.2 Architecture Components and Connections

1. Customer Website

   The Customer Website provides the main browser-based interface for customers. Customers can use this interface to browse movies, view showtimes and reviews, check seat availability, select tickets, make payments, access their accounts, and submit feedback. The website sends customer requests to the Application Server and displays the information returned by the system.

2. Self-Service Terminal

The Self-Service Terminal provides an in-person interface at each theater location. It allows customers to view movies and showtimes, select available tickets, complete payments, and receive printed or digital tickets. The terminal connects to the same Application Server and Central Database as the Customer Website. This ensures that online customers and customers at the theater receive the same movie, showtime, and ticket-availability information.

3. Administrator Portal

   The Administrator Portal is a restricted interface for authorized theater employees. Administrators can use it to manage movies, showtimes, auditoriums, seat availability, reviews, discounts, and customer purchasing problems. The system must verify the administrator's login information and permission level before allowing changes. Administrator actions are also recorded in the audit log.

4. Application Server

   The Application Server receives requests from the Customer Website, Self-Service Terminal, and Administrator Portal. It applies the system's business rules and directs each request to the correct service. It also controls communication between the user interfaces, Central Database, and outside systems. Examples include checking ticket limits, validating showtime dates, applying discounts, creating seat holds, and preventing unauthorized administrative actions.

5. Movie and Showtime Service

   The Movie and Showtime Service manages movie information, theater locations, auditoriums, schedules, and available formats such as 2D, 3D, and IMAX. It retrieves movie and showtime records from the Central Database and returns them to the customer interfaces. It also checks for scheduling conflicts when an administrator creates or updates a showtime.

6. Ticket and Seat Service

   The Ticket and Seat Service manages ticket selection, seat availability, temporary seat holds, and finalized bookings. When a customer selects an assigned seat, the service places a five-minute hold on that seat. The service checks the latest seat status before finalizing the transaction to prevent the same seat from being sold twice. If multiple customers attempt to purchase the same seat, the seat is assigned to the customer whose payment is approved first.

7. Payment and Order Service

   The Payment and Order Service calculates the subtotal, discounts, sales tax, and final purchase amount. It sends the payment request to the outside Payment Processor using a secure payment token. If the payment is approved, the service finalizes the booking,

changes the selected seats from held to sold, and generates a booking reference number and QR code. If the payment is declined, the booking is not completed and the customer is not charged.

8. Account and Loyalty Service

   The Account and Loyalty Service manages optional customer accounts, login information, purchase history, preferred language, and loyalty points. Guest customers can still purchase tickets without creating an account. Sensitive passwords must be stored in hashed form, and complete credit or debit card numbers will not be stored by the system.

9. Review and Feedback Service

   The Review and Feedback Service manages externally collected movie reviews and feedback submitted by customers. It receives review information from supported outside sources and connects each review to the correct movie. It also validates and sanitizes customer feedback before storing it. Verified feedback with a rating below two stars is marked as high priority and creates a notification for an administrator.

10. Notification Service

    The Notification Service sends purchase confirmations and digital tickets to customers by email. The confirmation includes the booking reference, showtime information, seat information, and QR code. The service may also send administrative notifications for high-priority customer feedback or important system events.

11. Central Database

    The Central Database stores information used across all theater locations. This includes customer accounts, movies, theaters, auditoriums, showtimes, seats, bookings, payments, discounts, reviews, feedback, and audit logs. The Customer Website and Self-Service Terminals use the same stored information so ticket availability remains consistent. Completed transactions and administrative changes are saved in the database.

12. External Systems

    The system communicates with three main types of outside services. The Payment Processor authorizes credit card, PayPal, and Bitcoin transactions. The Email Service delivers confirmations and digital tickets. External Review Services provide movie ratings or review information from sources such as Rotten Tomatoes or IMDb. If review information is unavailable, customers must still be able to browse movies and purchase tickets.

13. Component Connections

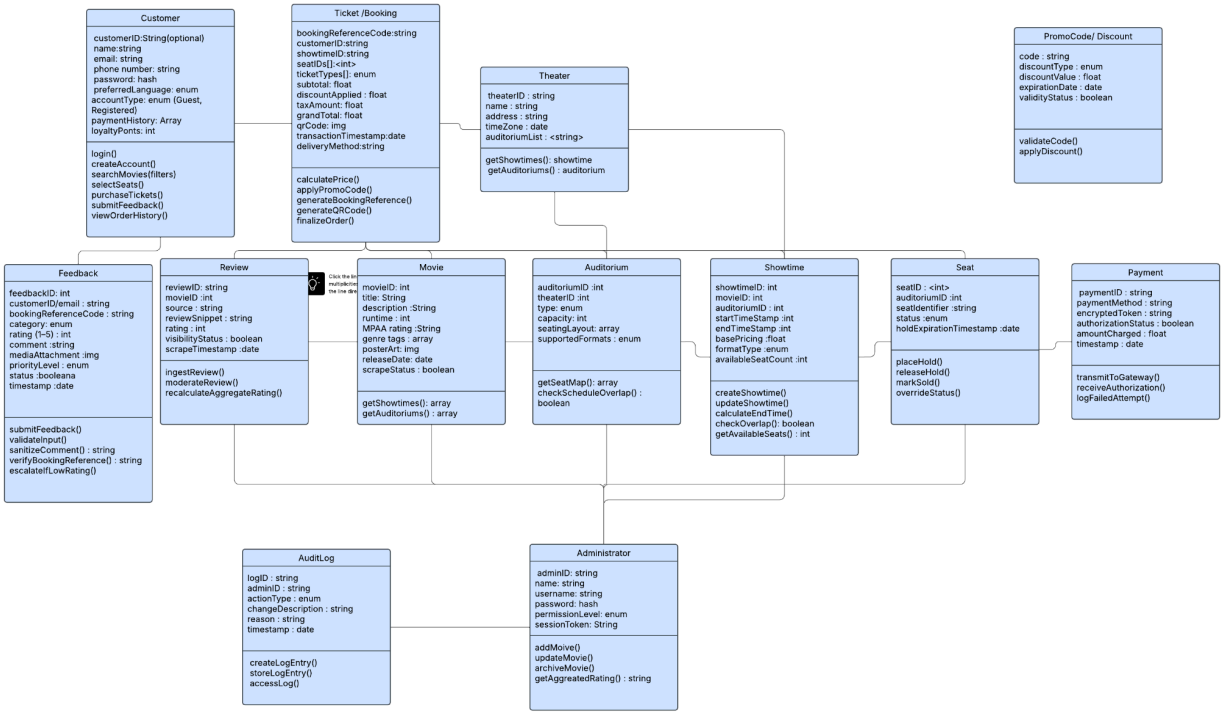
The Customer Website, Self-Service Terminal, and Administrator Portal send requests to the Application Server. The Application Server forwards each request to the appropriate internal service. Internal services read from or write to the Central Database as required. The Payment and Order Service communicates with the external Payment Processor, the Notification Service communicates with the Email Service, and the Review and Feedback Service communicates with External Review Services.

A typical ticket purchase begins when a customer selects a movie, showtime, and seat through the Customer Website or Self-Service Terminal. The request travels through the Application Server to the Ticket and Seat Service, which checks availability and creates a temporary hold. The Payment and Order Service then sends the payment request to the Payment Processor. After approval, the booking and seat status are saved in the Central Database, and the Notification Service sends the customer a confirmation.

6.3 UML Class Design

The UML class design identifies the major classes by the Movie Ticket Star system and shows how they relate to each other. Each class contains attributes representing the information it stores and operations representing the actions it performs. The class design is based on the functional requirements, use cases, and classes and objects identified in this SRS.

6.3.1 UML Class Diagram



6.3.2 Class Descriptions

The classes below correspond to the UML class diagram in 6.3.1 and the classes and objects defined in Section 3.4. Each description gives the purpose of the class and its main relationships to other classes.

1. **Customer** The Customer class represents a person who uses the Customer Website or a Self-Service Terminal. A customer is either a guest or a registered user, as set by the accountType attribute. Registered customers have a customerID, a hashed password, a preferred language, payment history, and loyalty points. Guests are not required to create an account. A customer searches for movies, selects seats, purchases tickets, and submits feedback. One customer may have many Ticket/Booking records and may submit many Feedback records.
2. **Administrator** The Administrator class represents an authorized theater employee who uses the Administrator Portal. Each administrator has a permissionLevel and a sessionToken. The system checks both before allowing any change. Administrators manage movies, showtimes, auditoriums, seat availability, and review moderation. An administrator may override the status of a seat that has already been sold, but only through a confirmation step. Every administrative change creates an AuditLog entry, so one administrator is associated with many AuditLog records.
3. **Movie** The Movie class stores the information shown to customers about a film, including its title, description, runtime, MPAA rating, genre tags, poster art, and release date. The scrapeStatus attribute shows whether review data has been collected for the film. A newly added movie is marked pending until reviews are collected. One movie can have many Showtimes and many Reviews. Its getAggregatedRating() operation combines the visible xReview ratings into the score shown on the movie card.
4. **Theater** The Theater class represents one of the theater locations. It stores the theater's name, address, time zone, and list of auditoriums. All theaters share the Central Database but are partitioned by theaterID. One theater contains many Auditoriums and can return the showtimes scheduled at that location.
5. **Auditorium** The Auditorium class represents an individual theater room. It stores the room's capacity, seating layout, and supported formats such as 2D, 3D, and IMAX. An Auditorium belongs to one Theater, contains many Seats, and hosts many Showtimes. The checkScheduleOverlap() operation prevents two showtimes from being scheduled in the same auditorium at overlapping times.
6. **Showtime** The Showtime class represents one scheduled screening of a movie in an auditorium. It stores the start time, the end time (start time plus the configurable buffer), the base ticket price, the format, and the number of available seats. Each Showtime is linked to exactly one Movie and one Auditorium. It is the basis for customer ticket purchases, so one Showtime can be linked to many Ticket/Booking records. Its seat availability is used to label screenings as available, nearly sold out, or sold out.
7. **Seat** The Seat class represents a single seat in an auditorium. Its status is one of Available, Held, Sold, Maintenance, or VIP. When a customer selects a seat, placeHold() reserves it for five minutes and records a holdExpirationTimestamp. If the timer expires or the customer leaves, releaseHold() makes the seat available again. After payment

approval, markSold() changes the status to Sold. The overrideStatus() operation is restricted to administrators. This class supports the requirement that the same seat is never sold to more than one customer.

8. **Ticket / Booking** The Ticket/Booking class represents one completed or in-progress purchase. It stores the booking reference code, the customer (optional for guests), the showtime, the selected seats, the ticket types, the subtotal, discounts, tax, grand total, QR code, and delivery method. It calculates the price from the ticket quantities and category prices, applies any valid promo code, adds local sales tax, and generates the booking reference and QR code once payment is approved. A booking is linked to one Showtime, one Payment, and at most one PromoCode/Discount. It cannot contain more than 20 tickets.
9. **PromoCode / Discount** The PromoCode/Discount class represents a discount that can be applied to a purchase. Its discountType is Student, Military, Senior, or Promotional, and each discount has a value, an expiration date, and a validity status. The validateCode() operation checks whether a code is recognized and still valid, and applyDiscount() reduces the booking subtotal. If a code is invalid or expired, the order is recalculated without the discount and the customer sees an error message.
10. **Payment** The Payment class represents the payment attempt for a booking. It records the payment method (credit card, PayPal, etc), an encrypted payment token, the authorization status, the amount charged, and a timestamp. It sends the token and total to the external Payment Processor and receives the approval or refusal. Complete card numbers are never stored. Declined attempts are recorded by logFailedAttempt(), and a declined payment does not complete the booking or charge the customer.
11. **Review** The Review class represents a rating or critic quote collected from an external source such as Rotten Tomatoes or IMDb. It stores the source, review snippet, rating, visibility status (Published, Hidden, or Flagged), and the time it was collected. Each Review belongs to one Movie. Administrators can hide or edit a review through moderation, which marks it as overridden and triggers recalculation of the movie's aggregate rating. If no review data is available, the movie card displays a placeholder such as "Review Pending."
12. **Feedback** The Feedback class represents feedback submitted by a customer about the theater, service, or website. It stores a category, a 1–5 rating, an optional comment, an optional image attachment, an optional customer ID or email, and an optional booking reference code. The class validates the required fields, sanitizes comments to prevent code injection, and verifies the booking reference code when one is provided. Verified feedback rated below two stars is marked as high priority, and an alert is sent to administrators. Submissions are limited to three per IP address in a 24-hour period.
13. **AuditLog** The AuditLog class records each administrative action. Each entry stores the administrator's ID, the type of action, a description of the change, the reason given, and a timestamp. Entries are created whenever an administrator changes showtimes, seats, movies, or reviews, which provides a traceable record of who changed what and why. Each AuditLog entry belongs to one Administrator.

6.4 Development Plan and Timeline

The Movie Ticket Star system will be developed using the waterfall life cycle model. Under this model, the group will complete the requirements, design, implementation, testing, and final documentation stages in order. Major changes to an earlier stage should be reviewed and approved before they are added to the project. The client expects the initial prototype to be completed within four months.

6.4.1 Partitioning of Tasks

The work required to develop the Movie Ticket Star system will be divided into the following tasks:

1. Requirements Analysis

   The group will collect the client's requirements and document the system's functional requirements, non-functional requirements, interfaces, use cases, classes, and constraints in the SRS.

2. Software Architecture Design

   The group will identify the system's major components, including the user interfaces, application services, database, and external services. The group will create a software architecture diagram and describe the responsibilities and connections of each component.

3. UML Class Design

   The group will create a UML class diagram showing the major system classes, their attributes, operations, and relationships. The written class descriptions will provide additional information about the purpose of each class.

4. Database Design

   The group will design the central database needed to store customers, movies, theaters, auditoriums, showtimes, seats, bookings, payments, discounts, reviews, feedback, and audit logs. The database design must support all theater locations while keeping seat and ticket information consistent.

5. User-Interface Development

   The group will develop the Customer Website, Self-Service Terminal interface, and Administrator Portal. The interfaces will allow each type of user to access the functions authorized for that user.

6. Application-Service Development

The group will implement the main system services for movie and showtime management, ticket and seat selection, payment and order processing, customer accounts, loyalty points, reviews, feedback, notifications, and administration.

7. External-Service Integration

   The group will connect the application to the outside Payment Processor, Email Service, and supported External Review Services. These connections will be tested to ensure that an outside-service failure does not prevent unrelated parts of the system from operating.

8. System Testing

   The group will test individual components and complete system functions. Testing will include ticket purchasing, seat holds, concurrent seat selection, payment approval and rejection, administrator authorization, account security, feedback submission, and ticket delivery.

9. Final Review and Documentation

   The group will review the completed system and documentation for accuracy, consistency, formatting, and traceability. Any approved corrections will be made before the final project is submitted.

The group will connect the application to the outside Payment Processor, Email Service, and supported External Review Services. These connections will be tested to ensure that an outside-service failure does not prevent unrelated parts of the system from operating.

6.4.2 Team Member Responsibilities

1. Dillon Holden
   Dillon will be responsible for the written Software Architecture description, organization of the new Software Design Specification sections, consistency between the architecture description and diagram, document formatting, and part of the final review and testing documentation.

2. Michael Farella
   Michael will be responsible for producing software design specification diagrams, reviewing the design process, implementing network and security features, helping verify the design against the existing SRS requirements, and participating in the final review and testing documentation.

3. Jhef Grover Quiambao
   Jhef will be responsible for developing the UML class diagram and class descriptions, defining class attributes, operations, data types, and relationships, assisting with the

integration of the Software Design Specification into the existing SRS, and participating in the final review and testing documentation.

4. Zhoubin Nazari Boukani
   Zhoubin will be responsible for the database design and data dictionary, defining the user interface and screen layouts for the ticketing workflow, helping verify the design against the existing SRS requirements, and participating in the final review and testing documentation.

6.4.3 Estimated Timeline

1. Weeks 1–2: Requirements Gathering and Analysis

   The group will interview the client, identify the system's expected functions, and document the functional and non-functional requirements.

2. Weeks 3–4: Requirements Specification

   The group will complete and review the SRS, including the external interfaces, use cases, classes and objects, system constraints, and analysis models.

3. Weeks 5–6: Software Design

   The group will create the Software Architecture diagram, UML class diagram, component descriptions, class descriptions, and database design.

4. Weeks 7–10: System Implementation

   The group will develop the customer interfaces, administrator interface, application services, and Central Database. Core ticket selection and purchasing functions will be completed before optional or supporting features.

5. Weeks 11–12: External-Service Integration

   The group will connect and test the Payment Processor, Email Service, and External Review Services.

6. Weeks 13–14: System Testing

   The group will perform component, integration, and system testing. Defects involving payments, duplicate seat sales, account security, and ticket delivery will receive the highest priority.

7. Week 15: Corrections and Final Review

The group will correct identified defects and verify that the completed system satisfies the requirements in the SRS.

8. Week 16: Final Documentation and Submission

   The group will complete the final project report, update diagrams and documentation, verify the GitHub repository, and prepare the final prototype for submission.

7.1 Overview

This plan defines how Group 4 will verify that the Movie Ticketing System meets the requirements in SRS version 1.1.

The system is a browser-based ticketing platform. Customers can browse movies and showtimes, select seats, apply discounts, pay, and receive tickets. Administrators manage showtimes, seating, reviews and audit logs through a separate portal. Both access paths share one central database so availability stays consistent.

Item	Detail
Project	Movie Ticketing System (Movie Ticket Star)
Course	CS 250 – Introduction to Software Systems, Dr. Gus Hanna
Team	Group 4: Michael Farella, Jhef Grover Quiambao, Dillon Holden, Zhoubin Nazari Boukani
Source documents	SRS v1.1; test case workbook (G4CS250MovieTicketingTestCases)
Plan date	2026-10-22

7.2 Scope and Objectives

Testing focuses on the core purchase path and the rules that protect it: pricing, seat holds, payment, ticket limits, login and administrator showtime management.

Objectives

- Confirm that a customer can complete a purchase from movie selection to ticket confirmation.
- Confirm that no seat is sold twice and that a failed payment never finalizes a booking.
- Confirm that business rules hold: 20-ticket limit, promo code validation, price calculation, 5-minute seat hold.
- Confirm that only valid administrator actions change showtimes, without scheduling conflicts.
- Record every result so defects can be traced back to an SRS requirement.

In scope

- Ticket selection and purchase (SRS 3.2.1): seat hold, price calculation, promo codes, payment authorization, order finalization, digital ticket email.
- Customer login and guest checkout (SRS 3.3.1, 3.4.1).
- Administrator showtime creation and overlap check (SRS 3.2.3, 3.3.2).

- Class-level checks on PromoCode, Ticket/Booking and Seat (SRS 3.4.7 to 3.4.9).
- Error handling for declined payment and ticket-limit violations.

Out of scope for this cycle

- Film and review display, review scraping and moderation (SRS 3.2.2, 3.2.3 review items).
- Customer feedback collection (SRS 3.2.4).
- Performance, availability and portability targets in SRS 3.5 (500 concurrent users, 99% uptime, cross-browser matrix).
- Multi-language support (English, Spanish, Swedish), loyalty points, printed tickets and hardware terminal devices.

These exclusions are deferred, not dropped. Section 4.3 lists the SRS areas that still need cases.

7.3 Test Strategy

Testing runs bottom-up: unit-level checks on single methods first, then integration of services, then end-to-end workflows. Each case has an ID, component, priority, prerequisites, numbered steps, an expected result, and is executed manually unless noted.

Level	What it covers	Test cases
Unit / component	One class method in isolation: validateCode(), calculatePrice(), placeHold()	PromoCode_1, TicketPrice_1, SeatHold_1
Functional	One feature through the UI: login, ticket limit, guest checkout, admin showtime	CustomerLogin_1, TicketLimit_1, GuestCheckout_1, AdminShowtime_1
Integration	Several services plus an external one: payment gateway, email service, seat service	PaymentFailure_1, DigitalTicket_1
System / end-to-end	The full customer purchase workflow	CompletePurchase_1
Regression	Re-run all P0 cases after every defect fix and before the final submission	All 10
Acceptance	Walk-through of the purchase and admin use cases by the team with the instructor's SRS as the checklist	Use Cases 1 and 2

Approach

- **Priority:** all current cases are P0, so they must all pass before the prototype is submitted. New cases added later will use P1 (important) and P2 (nice to have).
- **Test data:** one registered customer, one administrator, one valid promo code, one expired promo code, a declined test card, and a showtime with at least 21 open seats.
- **External services:** payment and email use sandbox or test accounts. No real card numbers are used, and complete card numbers are never stored (SRS 3.5.4).

- **Defects:** each failure is logged with the case ID, actual result and severity. Payment, duplicate seat sale, account security and ticket delivery defects are fixed first, as the SRS timeline requires.
- **Status values:** Not Run, Pass, Fail, Blocked. A case is only Pass when the actual result is recorded and matches the expected result.

7.4 Test Cases and Scenarios

Ten P0 cases cover the purchase path, seat handling, pricing, login and administration. Every case traces to an SRS section.

7.4.1 Test case summary

ID	Component	Priority	Summary	SRS reference	Owner
PromoCode_1	PromoCodeDiscount	P0	validateCode() accepts a valid, unexpired promo code	3.2.1.3(2), 3.4.9	Jhef Grover Quiambao
TicketPrice_1	TicketBooking	P0	calculatePrice() returns the correct subtotal and total	3.2.1.3(2), 3.4.8	Jhef Grover Quiambao
SeatHold_1	Seat	P0	placeHold() sets status to Held and stores the expiration time	3.2.1.3(1), 3.4.7	Jhef Grover Quiambao
CustomerLogin_1	Customer Login	P0	Registered customer logs in with valid credentials	3.4.1, 3.5.4(4)	Dillon Holden
TicketLimit_1	Ticket Booking	P0	System blocks more than 20 tickets in one transaction	1.2, 2.4, 3.2.1.2	Dillon Holden
GuestCheckout_1	Guest Checkout	P0	Customer buys tickets without a loyalty account	3.2.1.2, 3.3.1	Michael Farella
AdminShowtime_1	Administrator / Showtime	P0	Administrator creates a showtime with no schedule conflict	3.2.3.3(3), 3.3.2	Michael Farella
CompletePurchase_1	Complete Ticket Purchase	P0	Full flow from movie selection to confirmation	3.2.1, 3.3.1	Zhoubin Nazari Boukani
PaymentFailure_1	Booking / Payment / Seat	P0	Declined payment does not finalize booking or sell the seat	3.2.1.5(3), 3.5.2(4)	Zhoubin Nazari Boukani
DigitalTicket_1	Booking / Notification	P0	Completed purchase emails a digital ticket	3.2.1.4, 3.1.3	Zhoubin Nazari Boukani

7.4.2 Test case detail

ID	Pre-requisites	Steps	Expected result
PromoCode_1	A valid, unexpired promo code exists	1. Create or select a valid promo code. 2. Call validateCode(). 3. Check the returned result.	Returns true; the code is accepted
TicketPrice_1	Booking with known quantity and ticket prices	1. Create a booking with known prices. 2. Call calculatePrice(). 3. Compare with the expected total.	Subtotal and total match the expected prices and quantities
SeatHold_1	Selected seat is available	1. Select an available seat. 2. Call placeHold() with an expiration time. 3. Check seat status and timestamp.	Status is Held and the hold expiration is saved
CustomerLogin_1	Registered account with valid email and password	1. Open the system. 2. Select Login. 3. Enter email and password. 4. Submit.	Customer is authenticated and gains access to the account
TicketLimit_1	Movie and showtime with 21 or more tickets or seats available	1. Select a movie and showtime. 2. Try to select 21 tickets. 3. Continue to checkout.	System prevents purchase of more than 20 tickets
GuestCheckout_1	Movie, showtime and an available ticket or seat	1. Open the system without logging in. 2. Select movie and showtime. 3. Select ticket or seat. 4. Continue as guest. 5. Enter checkout details.	Customer reaches checkout with no account or login
AdminShowtime_1	Administrator account; movie and auditorium exist; no time overlap	1. Log in as administrator. 2. Open showtime management. 3. Select movie and auditorium. 4. Enter a valid start time. 5. Save.	New showtime is created and available in the system
CompletePurchase_1	System up; movie, showtime and seats available; payment service available	1. Open the system. 2. Select movie. 3. Select showtime. 4. Select ticket or seat. 5. Enter valid payment. 6. Confirm. 7. View confirmation.	Payment authorized, booking finalized, availability updated, confirmation received

ID	Pre-requisites	Steps	Expected result
PaymentFailure_1	Available seat; a test card that will be declined	1. Select movie and showtime. 2. Select an available seat. 3. Continue to payment. 4. Enter declined payment details. 5. Submit.	Payment rejected, booking not finalized, seat not permanently sold
DigitalTicket_1	Valid email; movie and showtime; payment and email services up	1. Complete a valid purchase. 2. Choose digital delivery. 3. Enter a valid email. 4. Finalize. 5. Check the inbox.	Booking completes and a digital ticket or confirmation is emailed

Status note. The workbook's Status column reads Pass for all 10 cases, but the Actual Result column reads Not yet executed. This plan treats every case as Not Run until a tester records an actual result.

7.5 Resources

Four team members share the testing, each owning the cases listed in section 4.1.

Role	Person	Responsibility
Tester: pricing, promo, seat hold	Jhef Grover Quiambao	PromoCode_1, TicketPrice_1, SeatHold_1; class diagram checks
Tester: login, ticket limit	Dillon Holden	CustomerLogin_1, TicketLimit_1; test documentation
Tester: guest checkout, admin	Michael Farella	GuestCheckout_1, AdminShowtime_1; network and security checks
Tester: purchase, payment, email	Zhoubin Nazari Boukani	CompletePurchase_1, PaymentFailure_1, DigitalTicket_1; test data and database
Reviewer	Dr. Gus Hanna	Approves the plan and reviews results

Environment and tools

- A test build of the website and administrator portal with a copy of the central database, kept separate from any real data.
- A current desktop browser (Chrome, Safari, Firefox); the SRS requires a web browser, not a mobile app.
- Sandbox accounts for the payment gateway and email service, plus a test inbox for ticket emails.
- The test case workbook for recording actual results; a shared defect list; the project GitHub repository.

7.6 Risk Assessment and Mitigation

The biggest risks sit around outside services and seat concurrency, because both are hard to reproduce on demand.

Risk	Impact	Mitigation
Payment gateway or email sandbox is unavailable	PaymentFailure_1 and DigitalTicket_1 are blocked	Request sandbox accounts early; keep a stub gateway and a test inbox as fallback
Concurrent seat sales are hard to reproduce	Double-booking bugs go unseen	Add a two-session concurrency case (section 4.3) and run it in two browsers at once
Late defects in payments or seat holds	Schedule slips past week 16	Test the purchase path first; fix payment and seat defects before anything else
Status and results are recorded inconsistently	Reported pass rate is wrong	Record the actual result before setting Status; the workbook currently shows Pass with no actual result
Requirements change after the plan is approved	Cases become outdated	Review the cases whenever the SRS change process approves a change
Small team with a shared coursework load	Cases are executed late	Each tester owns a fixed set of cases; progress is checked at every team meeting

7.7 Success Criteria

Testing succeeds when every P0 case has a recorded actual result and passes, with no open defect in the purchase path.

- **Pass rate:** 100% of P0 cases pass; at least 90% of any P1 cases pass.
- **Execution:** every case in section 4 has been run at least once and has an actual result recorded.
- **Defects:** no open defect that lets a seat be sold twice, finalizes a booking after a declined payment, or exposes account or payment data.
- **Regression:** all P0 cases pass again after the last defect fix.
- **Traceability:** each case maps to an SRS section, and the gaps in section 4.3 are either covered or accepted by the team as deferred.

Entry criteria: SRS v1.1 approved, test build deployed, test accounts and data ready.

Exit criteria: the success criteria above are met and the instructor has reviewed the results report.

7.8 Deliverables and Sign-off

- This test plan.
- The test case workbook with actual results, status and executor filled in for every case.
- A defect list with severity and fix status.
- A test results report: pass rate, open defects, and any deferred SRS areas.
- Regression results after the final fix.

Signature	Printed name	Title	Date
	Michael Farella, Jhef Grover Quiambao, Dillon Holden, Zhoubin Nazari Boukani	Software Eng.	
	Dr. Gus Hanna	Instructor, CS 250	

7.9 Github Repository

GitHub Repository:
https://github.com/JhefQuiambao/CS250Group4

8 Data Management Strategy

This section explains how Movie Ticket Star stores, protects, and recovers its persistent data. It describes the databases and other data stores, how the data is organized, how seat sales are kept consistent, how sensitive data is protected, how long data is kept, and why these choices were made over their alternatives.

8.1 Overview and Goals

The system stores customer accounts, booking and payment records, theater and movie information, showtimes and seat status, reviews, feedback, administrator actions, and image files. Some of this data is sensitive (passwords, contact details, payment-related records), and some is business critical (a sold seat must stay sold). The group chose a relational (SQL) data management strategy. The strategy is built to meet the following goals, each tied to an existing requirement:

- **Consistency.** A seat is never sold twice, and a booking, its payment, and its seat status are saved together or not at all (Section 3.5.2).
- **Confidentiality.** Passwords are hashed, full card numbers are never stored, and personal data is encrypted and access-controlled (Section 3.5.4).
- **Availability and recovery.** At least 99% availability per month, and purchasing restored within 15 minutes of an unexpected failure (Section 3.5.3).
- **Performance.** Listings and seat charts load within two seconds for 95% of requests with 500 simultaneous customers (Section 3.5.1).
- **Auditability.** Every administrative change leaves a permanent record (Section 3.2.3.3(7)).

- **Client constraint.** One database for all theaters, partitioned by theater (Sections 2.5 and 3.1.3).

8.2 Data Stores and Number of Databases

The design uses one database of record. It is supported by a standby copy, backups, and a file store for images. The table below lists each data store.

Data store	Proposed technology	Purpose	What it holds
Primary Central Database	PostgreSQL (relational, SQL)	The only place where data is created or changed. Serves the website, the self-service terminals, and the administrator portal.	All records listed in Section 8.3.
Hot Standby Replica	PostgreSQL streaming replication	Takes over if the primary fails. Read-only while the primary is healthy.	A continuously updated copy of the primary.
Backup Storage	Encrypted storage with nightly full backups and transaction log archiving	Restores the database after data loss or corruption, to any point in time.	Backups and the transaction log archive.
Media / File Storage	Application file storage (a cloud object store or a protected server folder)	Holds large files that do not belong in database rows.	Movie posters and feedback image attachments. The database keeps only the file path.

In short, there is one primary database and one standby copy of it. The standby is not a second set of data, because it holds exactly the same records. Backups and media files are not databases of record.

PostgreSQL is proposed because it provides the features this design depends on: transactions that are all-or-nothing, row-level locking, unique constraints, table partitioning by list, streaming replication, and point-in-time recovery from a transaction log. It is also free of license cost, which suits the $500,000 project budget. The design does not depend on PostgreSQL specifically. Any relational database with these features could be used, and it could run self-managed or as a managed cloud service.

8.3 Logical Organization of the Data

The data is divided into five schemas. A schema is a named group of tables inside the same database. The tables are grouped by who changes them and how sensitive they are, and each schema is written to by one owning service. Figure 8.3.1 shows the tables and their main relationships. The data dictionary that follows lists every column.

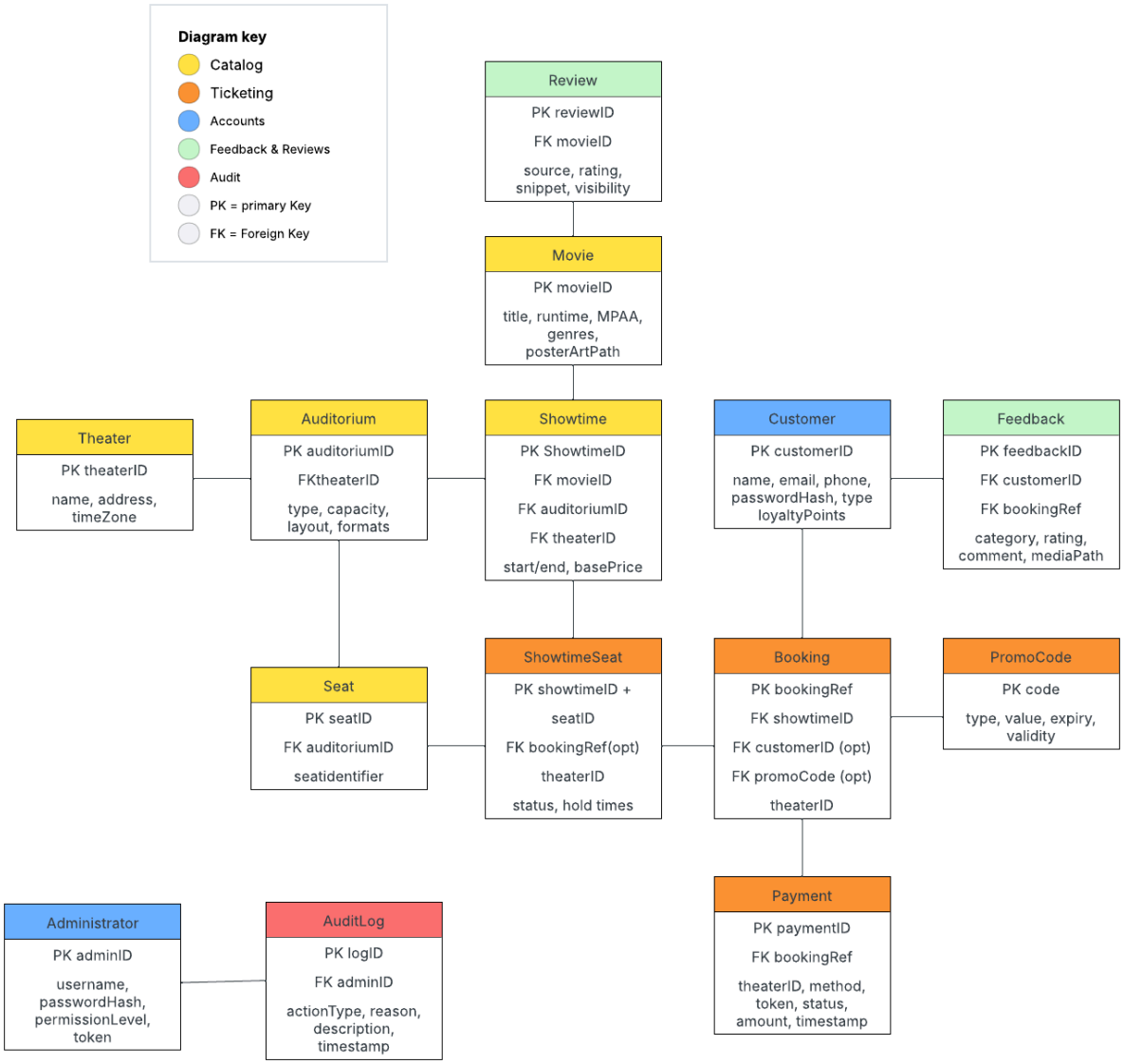


Figure 8.3.1: Logical data model showing tables, keys, relationships, and schemas

Data dictionary. Primary keys are marked PK, foreign keys FK, and the partition key is theaterID where noted.

Schema	Table	Columns	Notes
Accounts	Customer	customerID (PK), name, email (unique), phone, passwordHash, preferredLanguage, accountType (Guest or Registered), loyaltyPoints	Guests have no row. Payment history is derived from Booking and Payment.
Accounts	Administrator	adminID (PK), name, username (unique), passwordHash, permissionLevel, sessionTokenHash	Only hashes of passwords and session tokens are stored.
Catalog	Theater	theaterID (PK), name, address, timeZone	One row per location (20 at launch). The auditorium list is derived from Auditorium.

Catalog	Auditorium	auditoriumID (PK), theaterID (FK), type (Deluxe or Regular), capacity, seatingLayout, supportedFormats	
Catalog	Seat	seatID (PK), auditoriumID (FK), seatIdentifier (row and number)	The physical seat only. Status is not stored here.
Catalog	Movie	movieID (PK), title, description, runtime, mpaaRating, genreTags, posterArtPath, releaseDate, scrapeStatus	The poster image is in Media Storage.
Catalog	Showtime	showtimeID (PK), movieID (FK), auditoriumID (FK), theaterID (FK, partition key), startTimestamp, endTimestamp, basePrice, formatType	Partitioned. availableSeatCount is computed from ShowtimeSeat so it cannot drift.
Ticketing	ShowtimeSeat	showtimeID + seatID (composite PK), theaterID (partition key), status (Available, Held, Sold, Maintenance, VIP), holdExpiration, holdSessionID, bookingRef (FK, nullable)	Partitioned. One row per seat per showtime, created with the showtime. The unique key prevents double-selling.
Ticketing	Booking	bookingRef (PK), theaterID (partition key), showtimeID (FK), customerID (FK, empty for guests), promoCode (FK, nullable), contactEmail, ticketTypes, subtotal, discountApplied, taxAmount, grandTotal, qrCode, deliveryMethod, createdAt	Partitioned. contactEmail lets guests receive digital tickets. The 20-ticket limit is enforced by the Application Server.
Ticketing	Payment	paymentID (PK), bookingRef (FK), theaterID (partition key), paymentMethod, paymentToken, authorizationStatus, amountCharged, timestamp	Partitioned. Holds only the processor token, never a card number. Declined attempts are also recorded.
Ticketing	PromoCode	code (PK), discountType (Student, Military, Senior, Promotional), discountValue, expirationDate, validityStatus	Shared reference data, not partitioned.
Feedback and Reviews	Review	reviewID (PK), movieID (FK), source, reviewSnippet, rating, visibilityStatus, scrapeTimestamp	Collected from external review sources. Administrators can hide a review.
Feedback and Reviews	Feedback	feedbackID (PK), customerID (FK, nullable), email (nullable), bookingRef (FK, nullable), category, rating (1 to 5), comment, mediaPath, priorityLevel, status, timestamp	Comments are sanitized before storing. The image is in Media Storage.
Audit	AuditLog	logID (PK), adminID (FK), actionType, changeDescription, reason, timestamp	Insert-only. See Section 8.6

Access by service. Each backend service connects with its own database account. The table below shows what each account may do in each schema. W means read and write, R means read only, I means insert only, and a dash means no access. This keeps a fault or attack in one service from changing data it has no reason to touch.

Service account	Accounts	Catalog	Ticketing	Feedback and Reviews	Audit
Account / Loyalty	W	-	R	-	-
Movie / Showtime	-	W	R	R	I
Ticket / Seat	-	R	W	-	I
Payment / Order	R	R	W	-	-
Review / Feedback	-	R	R	W	I
Notification	R	R	R	R	-

8.4 Partitioning by Theater

The Showtime, ShowtimeSeat, Booking, and Payment tables are list-partitioned by theaterID, so each theater's rows are stored in their own partition (20 partitions at launch). This implements the client's statement that one database is partitioned across the different theaters. The other tables are not partitioned. Customers and administrators are not tied to one theater, and the catalog and reference tables are small.

Partitioning helps in a few ways. Firstly, almost every query in the purchase path names a theater so the database only reads the individual theaters partition. This supports the two-second response target. Partitioning by theater allows theaters to be added or removed without modifying the others.

Sizing assumption, to be confirmed with the client: about 10 auditoriums per theater, 150 seats per auditorium, and 5 showtimes per auditorium per day. That is about 7,500 ShowtimeSeat rows per theater per day, 150,000 per day across 20 theaters, and roughly 55 million per year. A single partitioned and indexed PostgreSQL server can handle this volume, which is the reason for the one-year purge rule in Section 8.4.

8.5 Consistency, Seat Holds, and Booking Transactions

The most important data rule is that a seat is never sold twice. The design enforces it in the database rather than relying only on application code. A purchase follows these steps:

1. Hold. When a customer selects seats, the Ticket/Seat Service runs one conditional update per seat: set status to Held, set holdExpiration to five minutes from now, and set holdSessionID, but only if the seat is Available or its previous hold has expired. If any selected seat is not updated, the whole transaction is rolled back and the customer is told the seat is taken. When two customers pick the same seat at the same instant, the database lock lets only one update succeed.

2. Expiry. A hold that passes its expiration time is treated as Available, and a scheduled job returns expired holds to Available every minute (releaseHold()).

3. Payment. The Payment/Order Service sends the secure token and total to the external Payment Processor. No database transaction is kept open while waiting for the processor.

4. Finalize. After approval, one transaction checks that every seat is still Held by this session and has not expired, changes the seats to Sold and links them to the booking, and inserts the Booking and Payment records. If any check fails, the transaction is rolled back, the authorization is voided, and the customer is not charged (Section 3.5.2(4)).

5. Decline. If the payment is declined, the Payment record is saved with a Declined status, the seats are released, and no booking is created.

6. Confirm. After the transaction commits, the Notification Service emails the confirmation. If the email fails, the booking stays valid and the email is retried.

Because each write is a conditional update on a single seat row, the default READ COMMITTED isolation level is enough, and the unique key on (showtimeID, seatID) is the final safeguard. When an administrator overrides a sold seat (Use Case 2), the status change, the refund flag on the booking, and the AuditLog entry are written in one transaction so they cannot be separated. All seat availability reads go to the primary database, so the website and terminals see a completed sale within the two-second target (Section 3.5.1(3)). The standby is not used for seat availability, because its copy can lag slightly behind.

8.6 Protecting Sensitive Data

Data	Sensitivity	Protection
Customer and administrator passwords	High	Stored only as a salted one-way hash (for example bcrypt or Argon2). Never stored or logged in plain text.
Payment card data	High	Never stored. Card details go directly to the external Payment Processor. The system keeps only the processor token, status, and amount (Section 3.5.4).
Administrator session tokens	High	Stored as a hash and invalidated at logout.
Customer contact details and purchase history	Medium to high	Encrypted at rest and in transit. Readable only by the services that need them (see the access table in 8.3).
Booking and payment records	Medium	Changed only by the Payment/Order account. Not deleted by the application.
Audit log	High (integrity)	Insert-only account. The application has no permission to update or delete entries.
Feedback comments and attachments	Medium	Comments are sanitized before storing. Attachments are stored outside the database under random file names, and are checked for type and size.
Movies, showtimes, reviews	Low	Public information. Changes limited to authorized administrators through the Movie/Showtime and Review services.

Three further rules apply to every data store. Services use separate least-privilege database accounts, and none uses an administrator account. The database accepts connections only from the application services, never directly from browsers or self-service terminals. All queries pass through the Data Access Layer and are parameterized.

8.7 Retention, Backup, and Recovery

The SRS does not state retention periods, so the values below are proposed by the group and must be confirmed with the client.

Data	Proposed retention	Reason
Bookings and payments (with the seats they sold)	7 years	Financial records, refunds, and disputes.
Customer account details	Until the customer closes the account, then deleted or anonymized	Limits how much personal data is kept. Booking totals are kept for financial records.

Unsold ShowtimeSeat rows for past showtimes	1 year, then deleted	Largest table by volume (Section 8.4) and no longer needed.
Seat holds	Minutes (five-minute expiry)	Temporary by design.
Feedback	2 years	Long enough to review trends.
Audit log	7 years	Matches the financial records the changes can affect.
Error logs (Section 3.5.5)	90 days	Enough time to diagnose problems.

Backup. A full backup runs nightly and the transaction log is archived continuously to encrypted Backup Storage. Backups are kept for 30 days, and the database can be restored to any point in time in that period. The group will perform a test restore before the prototype is delivered.

Failover. The Hot Standby Replica receives changes by streaming replication. If the primary fails, the standby is promoted, with a target of restoring ticket purchasing in under 15 minutes (Section 3.5.3(4)). Replication is asynchronous, so the target data loss is at most five minutes. If the client requires zero data loss, synchronous replication can be turned on for a small increase in purchase time.

Maintenance. Schema changes are applied during the scheduled maintenance window of at most two hours per month, with 24 hours notice to customers (Section 3.5.3).

8.8 Design Decisions

- **Why one database.** The client asked for one database for all theaters. It is also the best fit technically: a purchase touches seats, bookings, and payments, and these must commit together. One database can do that with an ordinary transaction. It is also the simplest option for a four-person team with a four-month prototype schedule. The standby and backups add availability without adding a second source of truth.
- **Why SQL.** The data is structured and highly related (a booking points to a showtime, a showtime to an auditorium, an auditorium to seats and a theater), the integrity rules are strict, and the main queries are known in advance. This is the case relational databases are built for.
- **Why five schemas.** Splitting by owning service and by sensitivity gives each service a clear boundary, lets permissions be set per schema, and lets the audit data be locked down more tightly than the rest. It does not require separate databases, so booking transactions stay simple. The schema boundaries also make a later split into separate databases easier if the system grows.
- **Why partition by theater.** It matches the client's wording, keeps each theater's queries small, and keeps archiving simple.
- **Why seat holds are stored in the database.** Holds are written to the same table as sold seats, so there is only one record of each seat's state and nothing can disagree after a

crash. The cost is a small amount of extra write traffic, which is acceptable for 500 simultaneous customers.

- **Why images are kept outside the database.** Images are large, rarely changed, and not needed for transactions. Keeping them in file storage keeps the database small, fast, and quick to back up.
- **Why a Data Access Layer.** It is one place to enforce least-privilege accounts, parameterized queries, and transaction handling, instead of repeating those rules in six services.

8.9 Alternatives and Tradeoffs

Technology alternatives. The table compares other technologies the group considered with the chosen PostgreSQL design.

Alternative	Advantages	Disadvantages	Why not chosen
Document (NoSQL) database, such as MongoDB	Flexible structure. Scales out easily. A booking and its seats can be read as one document.	Keeping a seat unique and updating seat, booking, and payment together needs multi-document transactions or extra application checks. Joins and reports are harder. The data is relational.	Strict consistency and relationships matter more here than schema flexibility.
In-memory key-value store (such as Redis) for seat holds	Built-in expiry suits five-minute holds. Very fast.	A second copy of seat state that can disagree with the database after a crash or timeout. One more component to run and secure.	Not needed at 500 simultaneous users. It is a good future option if hold traffic outgrows the database.
Distributed SQL database, such as CockroachDB	Scales across servers and regions. Automatic failover.	Higher cost and operational complexity. Features beyond what 20 theaters in one time zone need.	The extra capacity is not needed.
Embedded file database, such as SQLite	No setup and no server.	Not designed for many simultaneous users or network clients.	Cannot support 500 simultaneous customers across terminals and the website.

Organization alternatives. The table compares other ways the data could have been organized.

Alternative	Advantages	Disadvantages	Why not chosen
One database per theater (20 databases)	Strong isolation. A failure affects only one theater. Each can be scaled separately.	Conflicts with the client's one-database statement. Accounts and loyalty points span theaters. 20 backups and upgrades. Website and terminals must route to the right database. Cross-theater reports are hard.	Too much overhead, and not what the client asked for.

One database per service	Services deploy and scale independently. Very clear ownership.	A purchase spans seats, orders, and payments, so it would need distributed transactions or compensating steps. More servers to run.	Too complex for the team and schedule. Schemas give the ownership without distributed transactions.
Separate database for sensitive data (accounts and payments)	Stronger isolation of personal data. Can be hardened separately.	Bookings refer to customers and payments, so transactions and joins cross databases. Two systems to back up and fail over.	Separate schemas, accounts, and encryption give most of the benefit. The schema boundary allows a later split.
Separate database or external log service for the audit log	Tamper-resistant even against database administrators.	A second system. The audit entry no longer commits together with the change it records.	An insert-only schema is enough for the prototype. An external log is a possible upgrade.
One flat set of tables with no schemas	Simplest to start.	Weaker permission boundaries. Harder to split later.	Gives up ownership and audit protection for very little saving.
No partitioning, or sharding across servers	No partitioning is simplest. Sharding scales furthest.	Without partitions, large tables are slower to maintain and archive. Sharding makes cross-shard queries and transactions complex.	List partitioning by theater is simple, matches the client's wording, and keeps one server.

Tradeoffs accepted by the chosen design. The group accepts the following costs and has planned a mitigation for each.

- **One write point.** All writes go to one primary server, which limits growth to a larger server rather than more servers. Partitioning, indexing, and the standby reduce the risk, and a hold cache or distributed database remains an option if volume grows.
- **Shared database coupling.** A schema change can affect several services. Schema ownership, per-service accounts, and the change management process in Section 5 reduce this risk.
- **Added components.** The replica, backups, and file storage add cost and setup work, but they are needed to meet the availability and recovery requirements.
- **Holds as database writes.** Holds and expirations add write traffic to ShowtimeSeat. Short single-row transactions and the (showtimeID, status) index keep this small.
- **Asynchronous replication.** A failover could lose the last few minutes of changes. Synchronous replication can be enabled if the client prefers zero data loss over faster purchases.

A. Appendices

Appendices may be used to provide additional (and hopefully helpful) information. If present, the SRS should explicitly state whether the information contained within an appendix is to be considered as a part of the SRS's overall set of requirements.

Example Appendices could include (initial) conceptual documents for the software project, marketing materials, minutes of meetings with the customer(s), etc.

A.1 Appendix 1

A.2 Appendix 2